

Advanced Machine Learning

Lecture 5: Implementation of CNN, History, & DFT

Jens-Peter M. Zemke
zemke@tuhh.de

Institut für Mathematik
Lehrstuhl Numerische Mathematik
Technische Universität Hamburg

12 November 2024



Outline

Repetition

- Motivation

- Convolutions & pooling

- Feedforward & backpropagation

CNN

- Implementation

- History

- Convolutions & DFT

Outline

Repetition

Motivation

Convolutions & pooling

Feedforward & backpropagation

CNN

Implementation

History

Convolutions & DFT

Data we intend to treat in a satisfactory manner

High dimensional signals with several channels where order matters:

- ▶ audio signals (1D)
- ▶ images (2D)
- ▶ medical data, e.g. voxels (3D)
- ▶ voxels changing in time (4D), change of airflow around an airplane (4D)
- ▶ Cauchy stress tensor (9D), ...

Better treated by convolutional layers instead of fully connected layers:

- ▶ weights mostly zero (sparseness)
- ▶ take neighbouring information into account
- ▶ do not depend too much on absolute location, more on relative locations of data points

d D convolutions

Cross-correlation and convolution are defined for d D for any $d \in \mathbb{N}$,

$$(\mathbf{X} \star \mathbf{F})(n_1, \dots, n_d) := \sum_{\mathbf{i}} \mathbf{X}(n_1 + i_1, \dots, n_d + i_d) \cdot \mathbf{F}(i_1, \dots, i_d),$$

$$(\mathbf{X} * \mathbf{F})(n_1, \dots, n_d) := \sum_{\mathbf{i}} \mathbf{X}(n_1 - i_1, \dots, n_d - i_d) \cdot \mathbf{F}(i_1, \dots, i_d).$$

In 1D **stochastics** **cross-correlation** gives the **difference** of the probabilities $\mathbf{p}$ and $\mathbf{q}$ of two independent integer valued random variables X and Y ,

$$(\mathbf{p} \star \mathbf{q})_i = \sum_k p_{i+k} q_k = P[X - Y = i],$$

convolution gives the **sum** (compare w/ Cauchy product)

$$(\mathbf{p} * \mathbf{q})_i = \sum_k p_{i-k} q_k = P[X + Y = i].$$

Convolution as linear operation; structured weights

In 1D: for input $\mathbf{x} \in \mathbb{R}^n$ and kernel $\mathbf{f} \in \mathbb{R}^k$ we define

$$\mathbf{y} = \begin{pmatrix} y_1 \\ \vdots \\ y_k \\ \vdots \\ y_n \\ \vdots \\ y_{n+k-1} \end{pmatrix} = \mathbf{x} * \mathbf{f} = \mathbf{T}\mathbf{x} := \begin{pmatrix} f_1 & & & & \\ \vdots & \ddots & & & \\ f_k & \dots & f_1 & & \\ & \ddots & \ddots & \ddots & \\ & & f_k & \dots & f_1 \\ & & & \ddots & \vdots \\ & & & & f_k \end{pmatrix} \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix}, \quad \mathbf{T} \in \mathbb{R}^{(n+k-1) \times n}.$$

Python:

- ▶ `np.convolve(x, f, mode='full')` (default)
- ▶ `np.convolve(x, f, mode='valid')` only the middle part

The Toeplitz connection

In 2D: convolution / cross-correlation with **rotated kernel** $\mathbf{Y} = \mathbf{X} * \mathbf{F} = \mathbf{X} \star \text{np.rot90}(\mathbf{F}, 2)$, e.g.,

$$\begin{pmatrix} y_{1,1} & y_{1,2} \\ y_{2,1} & y_{2,2} \end{pmatrix} = \begin{pmatrix} x_{1,1} & x_{1,2} & x_{1,3} \\ x_{2,1} & x_{2,2} & x_{2,3} \\ x_{3,1} & x_{3,2} & x_{3,3} \end{pmatrix} * \begin{pmatrix} f_{1,1} & f_{1,2} \\ f_{2,1} & f_{2,2} \end{pmatrix} = \begin{pmatrix} x_{1,1} & x_{1,2} & x_{1,3} \\ x_{2,1} & x_{2,2} & x_{2,3} \\ x_{3,1} & x_{3,2} & x_{3,3} \end{pmatrix} \star \begin{pmatrix} f_{2,2} & f_{2,1} \\ f_{1,2} & f_{1,1} \end{pmatrix}$$

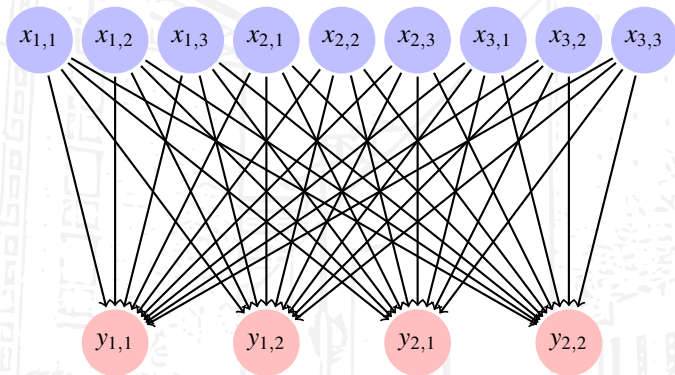
w/ a **block Toeplitz matrix with Toeplitz blocks**,

$$\begin{pmatrix} y_{1,1} \\ y_{1,2} \\ y_{2,1} \\ y_{2,2} \end{pmatrix} = \left(\begin{array}{ccc|ccc|ccc} f_{2,2} & f_{2,1} & 0 & f_{1,2} & f_{1,1} & 0 & 0 & 0 & 0 \\ 0 & f_{2,2} & f_{2,1} & 0 & f_{1,2} & f_{1,1} & 0 & 0 & 0 \\ \hline 0 & 0 & 0 & f_{2,2} & f_{2,1} & 0 & f_{1,2} & f_{1,1} & 0 \\ 0 & 0 & 0 & 0 & f_{2,2} & f_{2,1} & 0 & f_{1,2} & f_{1,1} \end{array} \right) \cdot \begin{pmatrix} x_{1,1} \\ x_{1,2} \\ x_{1,3} \\ \hline x_{2,1} \\ x_{2,2} \\ x_{2,3} \\ \hline x_{3,1} \\ x_{3,2} \\ x_{3,3} \end{pmatrix}.$$

```
from scipy.signal import convolve2d
Y = convolve2d(X, F, mode='valid')
```

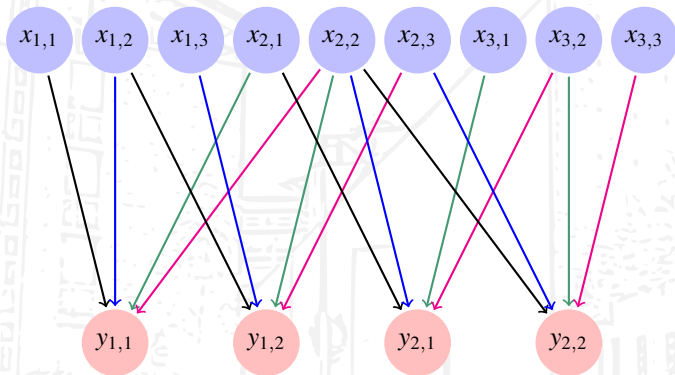
Weight sharing, pictorial understanding

Weight sharing: fully connected ($9 \times 4 = 36$)



Weight sharing, pictorial understanding

Weight sharing: fully connected ($9 \times 4 = 36$) vs. convolutional layer ($4 \times 4 = 16$ **shared**)



Pooling

Motivation in 1D: of interest points of **maximal de- or increase**. Compute maximum over small region after ReLU $\rightsquigarrow$ max pooling, data reduction

Generalizes to dD , we focus on 2D. Pooling is typically applied **after a convolutional layer**.

The most common pooling layers in 2D are given by

max pooling: mostly reduction of four neighbouring pixels to one that carries the maximal value.
For later convenience: **store “winning” indices** $\rightsquigarrow$ backpropagation

min pooling: basically like max pooling, only based on minimum. Again **storage of “winning” indices**

averaging: mostly reduction of four neighbouring pixels to one value that contains the **average of the four**

dropping: only every second, third, $\dots$ row and column is used, this is usually implemented as part of the convolution (**convolution with strides**) as there is no need to compute quantities that are dropped afterwards

Summary of changes for convolutions

To summarize changes in feedforward, backpropagation, & update with convolutions:

Feedforward: we use a completely overlapping convolution with a kernel F , e.g.,

1D: `y = np.convolve(a, f, mode='valid')`

2D: `Y = convolve2d(A, F, mode='valid')`

Backpropagation: we use a full convolution with the flipped kernel, e.g.,

1D: `np.convolve(delta, f[::-1], mode='full')`

2D: `convolve2d(Delta, F[::-1,::-1], mode='full')`

Kernel update: we use a completely overlapping convolution with either flipped activation or flipped δ vector (in which case we have to flip back), e.g.,

1D: `df = np.convolve(a[::-1], delta, mode='valid')`

2D: `dF = convolve2d(A[::-1,::-1], Delta, mode='valid')`

Pooling backpropagation

To obtain **backpropagation** for a **CNN with pooling**, we have to consider backpropagation through a pooling layer.

In **max pooling** we stored the values of the indices i where $a_i = \max_j a_j$. Suppose $i = 1$, then we can write

$$\max_j a_j = 1 \cdot a_1 + 0 \cdot a_2 + 0 \cdot a_3 + \dots$$

which has derivative 1 for a_1 and is zero elsewhere. The same holds true for **min pooling**.

For **averaging** k numbers we can write

$$\text{average}_j(a_j) = \frac{1}{k} \sum_{j=1}^k a_j,$$

thus the derivative is $1/k$ for all j .

Outline

Repetition

- Motivation
- Convolutions & pooling
- Feedforward & backpropagation

CNN

- Implementation
- History
- Convolutions & DFT

NCHW format

Minibatches of images, one to three channels, 2D-CNN: 4-tensor of images

$$\mathbf{A} = \mathbf{X} \in \mathbb{R}^{N \times C \times H \times W} \quad (\text{NCHW format})$$

N : minibatch size, C : number of channels, H : height, W : width

NCHW format

Minibatches of images, one to three channels, 2D-CNN: 4-tensor of images

$$\mathbf{A} = \mathbf{X} \in \mathbb{R}^{N \times C \times H \times W} \quad (\text{NCHW format})$$

N : minibatch size, C : number of channels, H : height, W : width

NumPy arrays: (long) vector in memory, last index fastest.

NCHW format

Minibatches of images, one to three channels, 2D-CNN: 4-tensor of images

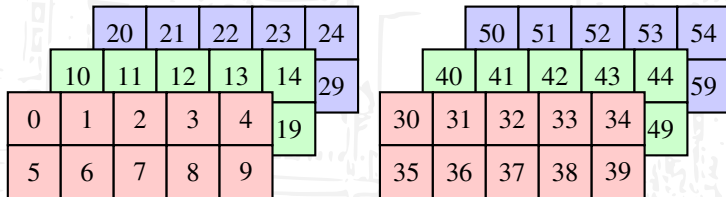
$$\mathbf{A} = \mathbf{X} \in \mathbb{R}^{N \times C \times H \times W} \quad (\text{NCHW format})$$

N : minibatch size, C : number of channels, H : height, W : width

NumPy arrays: (long) vector in memory, last index fastest.

Example ($N = 2$, $C = 3$, $H = 2$, $W = 5$):

```
A = np.arange(N * C * H * W).reshape(N, C, H, W)
```



CNN input formats

Formats:

NCHW / “channels first”: default in PyTorch, BAIR Caffe; available in TensorFlow 2.18.0

```
data_format='NCHW',  
tf.keras.backend.set_image_data_format('channels_first')
```

NHWC / “channels last”: TensorFlow 2.18.0 default `data_format='NHWC'`

```
tf.keras.backend.set_image_data_format('channels_last')
```

CHWN: used by Neon

Blocked layout: **nChw16c** on AVX512+ systems and **nChw8c** on SSE4.1+ systems [3]

CNN input formats

Formats:

NCHW / “channels first”: default in PyTorch, BAIR Caffe; available in TensorFlow 2.18.0

```
data_format='NCHW',  
tf.keras.backend.set_image_data_format('channels_first')
```

NHWC / “channels last”: TensorFlow 2.18.0 default `data_format='NHWC'`

```
tf.keras.backend.set_image_data_format('channels_last')
```

CHWN: used by Neon

Blocked layout: **nChw16c** on AVX512+ systems and **nChw8c** on SSE4.1+ systems [3]

Backends:

GPU $\rightsquigarrow$ **NVIDIA cuDNN** [1]: better use **NHWC**, **NCHW** available <https://docs.nvidia.com/deeplearning/performance/dl-performance-convolutional/index.html>

CPU $\rightsquigarrow$ **oneDNN** (formerly Intel(R) MKL-DNN / DNNL): better use **NCHW**
https://oneapi-src.github.io/oneDNN/dev_guide_understanding_memory_formats.html

Filters & convolutions

Use many 2D-filters to compute **several feature maps**, sum over all possible channels (or feature maps of previous layer) to obtain one feature map. **Filter bank** is also **4-tensor**:

$$\mathbf{F} \in \mathbb{R}^{M \times C \times f_h \times f_w}$$

M : number of filters, C : channels of input, f_h : height of filter, f_w : width of filter

Filters & convolutions

Use many 2D-filters to compute **several feature maps**, sum over all possible channels (or feature maps of previous layer) to obtain one feature map. **Filter bank** is also **4-tensor**:

$$\mathbf{F} \in \mathbb{R}^{M \times C \times f_h \times f_w}$$

M : number of filters, C : channels of input, f_h : height of filter, f_w : width of filter

The next **4-tensor** feature map $\mathbf{Z}$ is of shape (N, M, z_h, z_w) , where for a valid convolution $z_h = H - f_h + 1$, $z_w = W - f_w + 1$, and computed using

$$\mathbf{Z}[i, j, :, :] = b_j \cdot \text{np.ones}(z_h, z_w) + \sum_{k=1}^C \mathbf{A}[i, k, :, :] * \mathbf{F}[j, k, :, :], \quad \mathbf{A}_{\text{new}} = a(\mathbf{Z}).$$

Filters & convolutions

Use many 2D-filters to compute **several feature maps**, sum over all possible channels (or feature maps of previous layer) to obtain one feature map. **Filter bank** is also **4-tensor**:

$$\mathbf{F} \in \mathbb{R}^{M \times C \times f_h \times f_w}$$

M : number of filters, C : channels of input, f_h : height of filter, f_w : width of filter

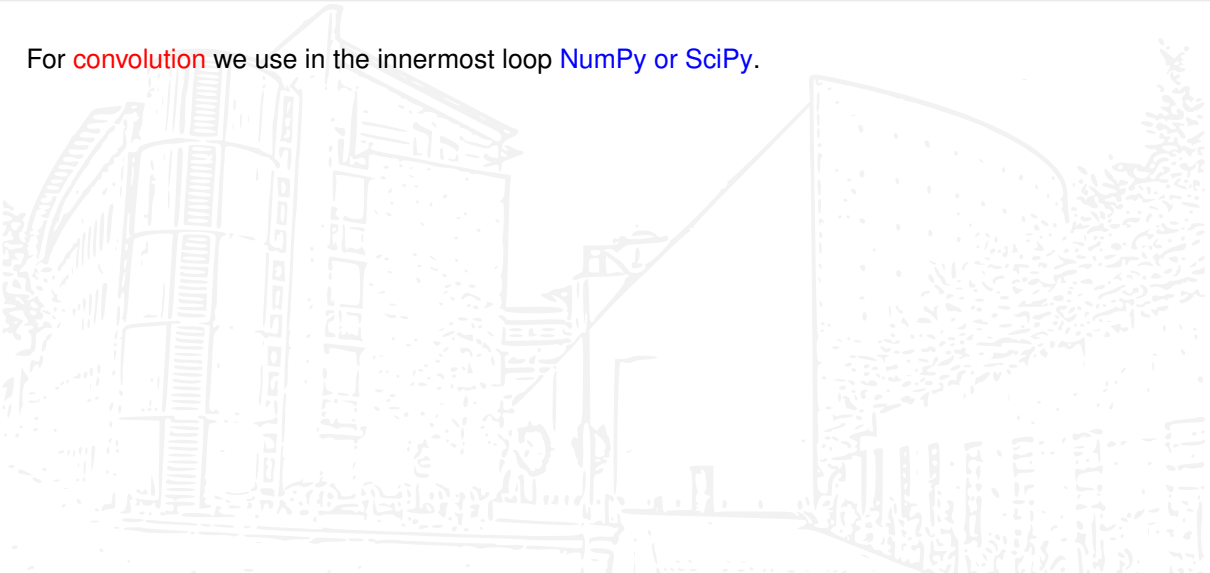
The next **4-tensor** feature map $\mathbf{Z}$ is of shape (N, M, z_h, z_w) , where for a valid convolution $z_h = H - f_h + 1$, $z_w = W - f_w + 1$, and computed using

$$\mathbf{Z}[i, j, :, :] = b_j \cdot \text{np.ones}(z_h, z_w) + \sum_{k=1}^C \mathbf{A}[i, k, :, :] * \mathbf{F}[j, k, :, :], \quad \mathbf{A}_{\text{new}} = a(\mathbf{Z}).$$

The **bias** is **constant per feature map**. The backpropagation has to be changed, the derivative is the **sum of all entries** of the corresponding Δ tensor, `db = np.sum(delta, axis=(0, 2, 3))`.

Simple implementation; pooling

For **convolution** we use in the innermost loop **NumPy** or **SciPy**.



Simple implementation; pooling

For **convolution** we use in the innermost loop **NumPy** or **SciPy**.

We mostly consider **max pooling based on 2×2 submatrices**. Max pooling maps an activation 4-tensor **A** of shape $(N \times C \times 2h(+1) \times 2w(+1))$ to a smaller 4-tensor of shape $(N \times C \times h \times w)$.

Simple implementation; pooling

For **convolution** we use in the innermost loop **NumPy** or **SciPy**.

We mostly consider **max pooling based on 2×2 submatrices**. Max pooling maps an activation 4-tensor $\mathbf{A}$ of shape $(N \times C \times 2h(+1) \times 2w(+1))$ to a smaller 4-tensor of shape $(N \times C \times h \times w)$.

Example: **max pooling** applied to a square matrix $\mathbf{A} \in \mathbb{R}^{m \times m}$ with even number of rows and columns $m = 2n$ using `reshape`. The call

```
A.reshape(n, 2, n, 2).max(axis=(1, 3))
```

gives the $n \times n$ matrix of maximal entries. If $m = 2n + 1$ is odd we use the first $2n$ elements.

Simple implementation; pooling

For **convolution** we use in the innermost loop **NumPy** or **SciPy**.

We mostly consider **max pooling based on 2×2 submatrices**. Max pooling maps an activation 4-tensor $\mathbf{A}$ of shape $(N \times C \times 2h(+1) \times 2w(+1))$ to a smaller 4-tensor of shape $(N \times C \times h \times w)$.

Example: **max pooling** applied to a square matrix $\mathbf{A} \in \mathbb{R}^{m \times m}$ with even number of rows and columns $m = 2n$ using `reshape`. The call

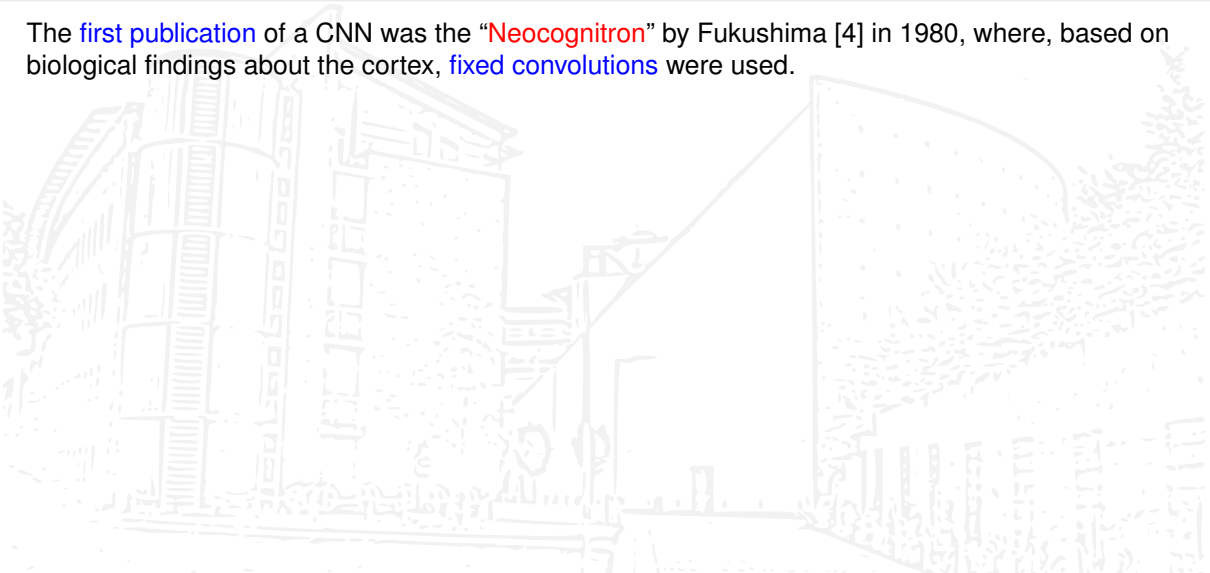
```
A.reshape(n, 2, n, 2).max(axis=(1, 3))
```

gives the $n \times n$ matrix of maximal entries. If $m = 2n + 1$ is odd we use the first $2n$ elements.

In the **exercises** we implement parts of a simple CNN, but w/o using a GPU we are **restricted to small CNN and datasets**.

The beginning

The **first publication** of a CNN was the “**Neocognitron**” by Fukushima [4] in 1980, where, based on biological findings about the cortex, **fixed convolutions** were used.



The beginning

The **first publication** of a CNN was the “**Neocognitron**” by Fukushima [4] in 1980, where, based on biological findings about the cortex, **fixed convolutions** were used.

Backpropagation in a CNN was first used by LeCun et al. [9] in 1989. The nets that were constructed by LeCun and coworkers have the names **LeNet-1** to **LeNet-5** and differ in size and number of convolutions.

The beginning

The **first publication** of a CNN was the “**Neocognitron**” by Fukushima [4] in 1980, where, based on biological findings about the cortex, **fixed convolutions** were used.

Backpropagation in a CNN was first used by LeCun et al. [9] in 1989. The nets that were constructed by LeCun and coworkers have the names **LeNet-1** to **LeNet-5** and differ in size and number of convolutions.

LeNet-5 by LeCun et al. [10] from 1998 was used on **industrial scale** for **deciphering handwritten digits** on checks and can be considered the first successful CNN. The pooling used was some sort of averaging, activation function was TanH.

The beginning

The **first publication** of a CNN was the “**Neocognitron**” by Fukushima [4] in 1980, where, based on biological findings about the cortex, **fixed convolutions** were used.

Backpropagation in a CNN was first used by LeCun et al. [9] in 1989. The nets that were constructed by LeCun and coworkers have the names **LeNet-1** to **LeNet-5** and differ in size and number of convolutions.

LeNet-5 by LeCun et al. [10] from 1998 was used on **industrial scale** for **deciphering handwritten digits** on checks and can be considered the first successful CNN. The pooling used was some sort of averaging, activation function was TanH.

Max pooling was introduced by Yamaguchi et al. [17] **for 1D in 1990** and by Weng, Ahuja, and Huang [16] **for 2D in 1993**.

The beginning

The first publication of a CNN was the “Neocognitron” by Fukushima [4] in 1980, where, based on biological findings about the cortex, fixed convolutions were used.

Backpropagation in a CNN was first used by LeCun et al. [9] in 1989. The nets that were constructed by LeCun and coworkers have the names LeNet-1 to LeNet-5 and differ in size and number of convolutions.

LeNet-5 by LeCun et al. [10] from 1998 was used on industrial scale for deciphering handwritten digits on checks and can be considered the first successful CNN. The pooling used was some sort of averaging, activation function was TanH.

Max pooling was introduced by Yamaguchi et al. [17] for 1D in 1990 and by Weng, Ahuja, and Huang [16] for 2D in 1993.

The success of CNN started when they were trained on GPUs using CUDA instead of CPUs, see, e.g., Ciresan et al. [2], the most prominent CNN is AlexNet by Krizhevsky, Sutskever, and Hinton [7, 8]. More recent and much deeper is ResNet by He et al. [5].

LeNet-5

LeNet-5 (Yann LeCun, Leon Bottou, Yosuha Bengio and Patrick Haffner, 1998, for digits on checks)

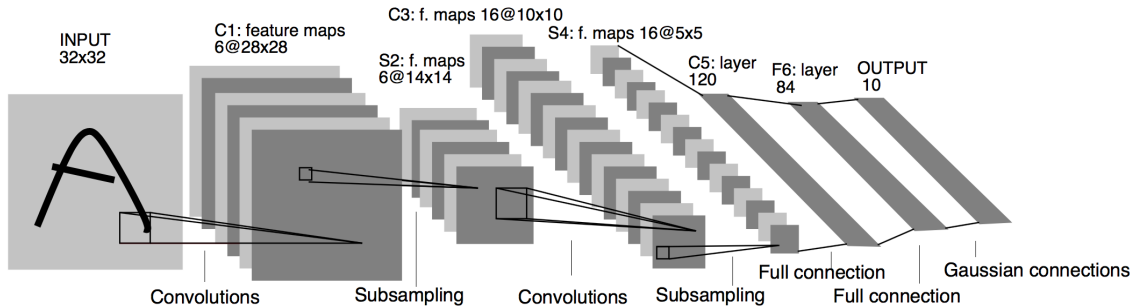


Image taken from LeCun et al. [10].

LeNet-5

LeNet-5 (Yann LeCun, Leon Bottou, Yosuha Bengio and Patrick Haffner, 1998, for digits on checks)

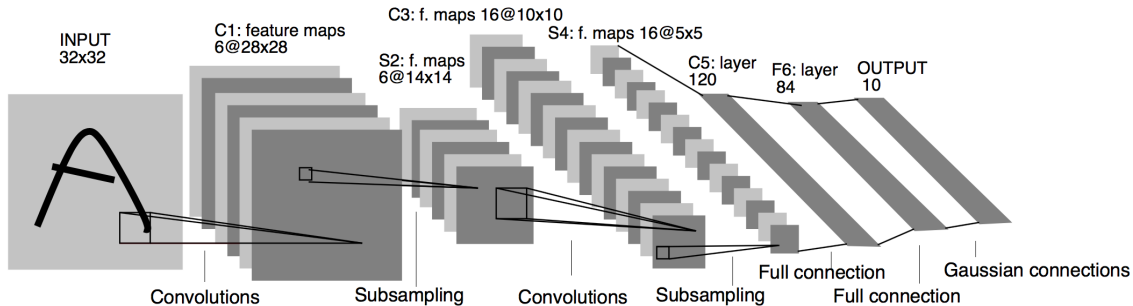


Image taken from LeCun et al. [10].

Used TanH, “averaging”, **three convolutional layers** (one uses 1×1 kernels), RBF with fixed centers. Counting convolution and subsampling as one layer, LeNet-5 has 5 layers (sic).

AlexNet

AlexNet (2012) by **Alex** Krizhevsky, Ilya Sutskever, Geoffrey E. Hinton, winner of ILSVRC (ImageNet Large Scale Visual Recognition Competition) Challenge in 2012

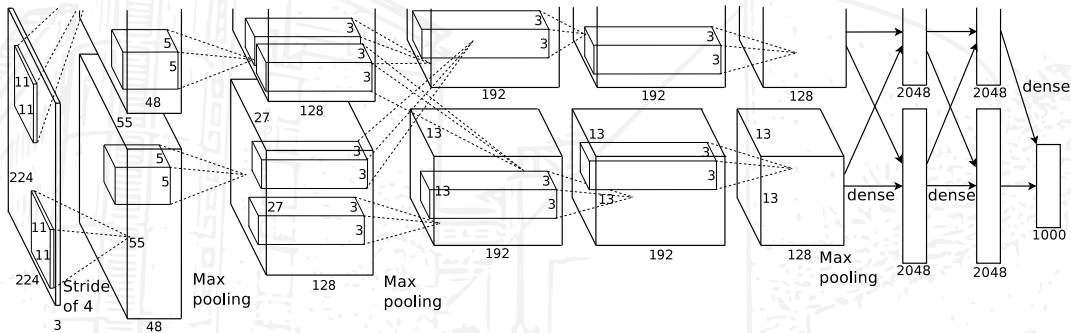


Image taken from Krizhevsky, Sutskever, and Hinton [7].

AlexNet

AlexNet (2012) by **Alex** Krizhevsky, Ilya Sutskever, Geoffrey E. Hinton, winner of ILSVRC (ImageNet Large Scale Visual Recognition Competition) Challenge in 2012

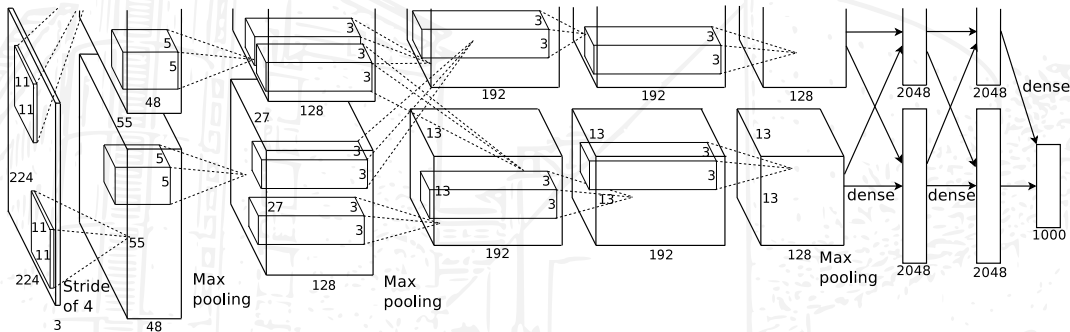
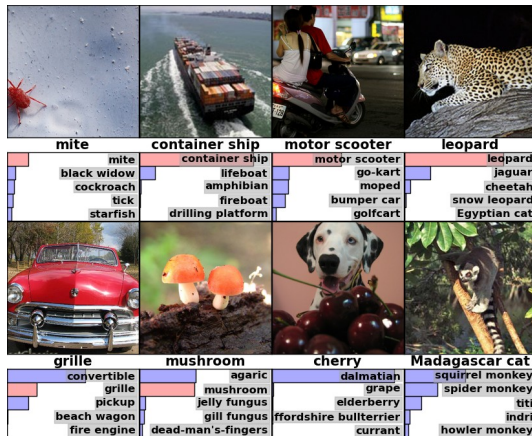


Image taken from Krizhevsky, Sutskever, and Hinton [7].

AlexNet has **8 layers**, **5 convolutional** followed by three dense layers. They introduced the **ReLU** activation function. Training on **two GPUs** for one week.

AlexNet – results

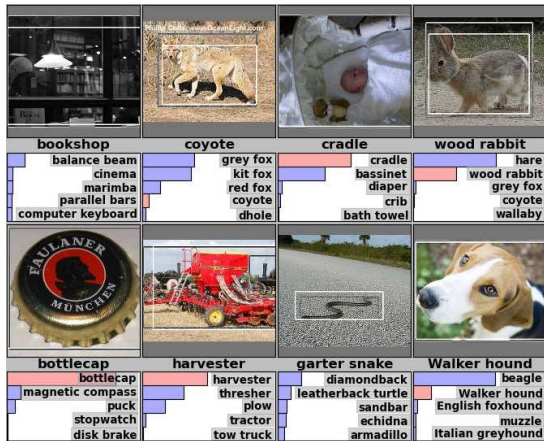
Validation classification



Slides taken from <http://www.image-net.org/challenges/LSVRC/2012/supervision.pdf>

AlexNet – results

Validation localizations

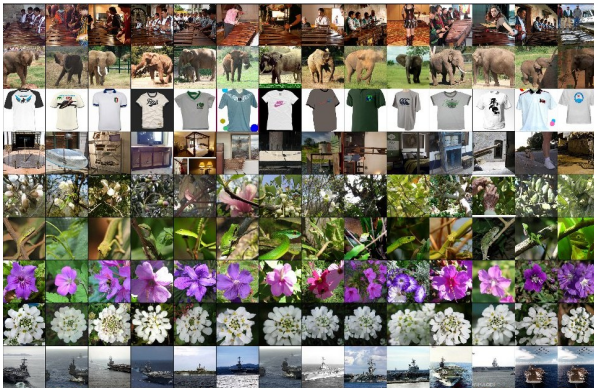


Slides taken from <http://www.image-net.org/challenges/LSVRC/2012/supervision.pdf>

AlexNet – results

Retrieval experiments

First column contains query images from ILSVRC-2010 test set, remaining columns contain retrieved images from training set.

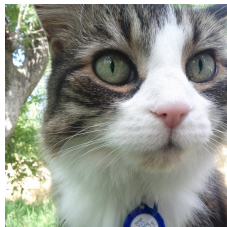


Slides taken from <http://www.image-net.org/challenges/LSVRC/2012/supervision.pdf>

AlexNet – treatment of input

Input representation

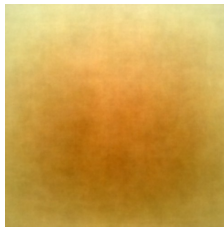
- Centered (0-mean) RGB values.



An input image (256x256)



Minus sign



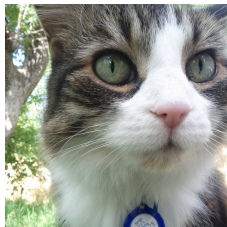
The mean input image

Slides taken from <http://www.image-net.org/challenges/LSVRC/2012/supervision.pdf>

AlexNet – treatment of input

Input representation

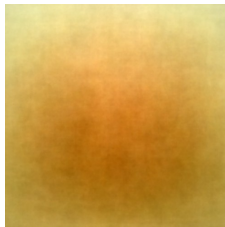
- Centered (0-mean) RGB values.



An input image (256x256)



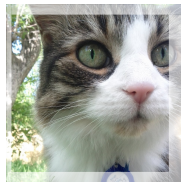
Minus sign



The mean input image

Data augmentation

- Our neural net has 60M real-valued parameters and 650,000 neurons
- It overfits a lot. Therefore we train on 224x224 patches extracted randomly from 256x256 images, and also their horizontal reflections.

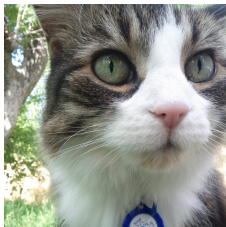


Slides taken from <http://www.image-net.org/challenges/LSVRC/2012/supervision.pdf>

AlexNet – treatment of input

Input representation

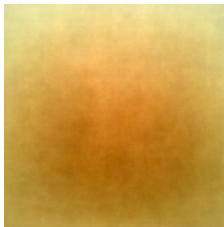
- Centered (0-mean) RGB values.



An input image (256x256)



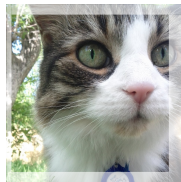
Minus sign



The mean input image

Data augmentation

- Our neural net has 60M real-valued parameters and 650,000 neurons
- It overfits a lot. Therefore we train on 224x224 patches extracted randomly from 256x256 images, and also their horizontal reflections.

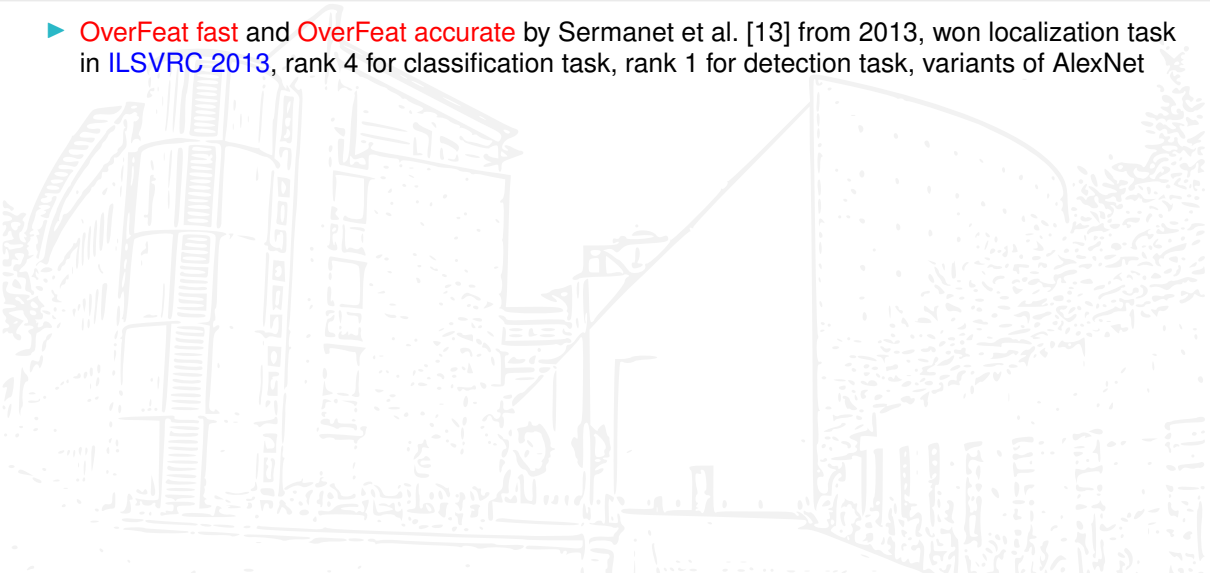


Slides taken from <http://www.image-net.org/challenges/LSVRC/2012/supervision.pdf>

Serious question: did they use **224** or maybe **227**? Did they work with **zero padding**?

Others

- ▶ **OverFeat fast** and **OverFeat accurate** by Sermanet et al. [13] from 2013, won localization task in **ILSVRC 2013**, rank 4 for classification task, rank 1 for detection task, variants of AlexNet



Others

- ▶ **OverFeat fast** and **OverFeat accurate** by Sermanet et al. [13] from 2013, won localization task in **ILSVRC 2013**, rank 4 for classification task, rank 1 for detection task, variants of AlexNet
- ▶ **VGG-16** by Simonyan and Zisserman [14], 13 convolutional layers, submitted to **ILSVRC 2014**. Developed by Oxford's Visual Geometry Group, weights and biases publically available, **replaces large kernels** (11×11 and 5×5) in AlexNet by several small 3×3 kernels.

Others

- ▶ **OverFeat fast** and **OverFeat accurate** by Sermanet et al. [13] from 2013, won localization task in **ILSVRC 2013**, rank 4 for classification task, rank 1 for detection task, variants of AlexNet
- ▶ **VGG-16** by Simonyan and Zisserman [14], 13 convolutional layers, submitted to **ILSVRC 2014**. Developed by Oxford's Visual Geometry Group, weights and biases publically available, **replaces large kernels** (11×11 and 5×5) in AlexNet by several small 3×3 kernels.
- ▶ **GoogLeNet (Inception) v1** by Szegedy et al. [15], 22 layers, 2014, winner of **ILSVRC 2014**. Contains **inception modules**: stacking different types of convolutions. Uses **1×1 kernels** for dimensionality reduction.

Others

- ▶ **OverFeat fast** and **OverFeat accurate** by Sermanet et al. [13] from 2013, won localization task in **ILSVRC 2013**, rank 4 for classification task, rank 1 for detection task, variants of AlexNet
- ▶ **VGG-16** by Simonyan and Zisserman [14], 13 convolutional layers, submitted to **ILSVRC 2014**. Developed by Oxford's Visual Geometry Group, weights and biases publically available, **replaces large kernels** (11×11 and 5×5) in AlexNet by several small 3×3 kernels.
- ▶ **GoogLeNet (Inception) v1** by Szegedy et al. [15], 22 layers, 2014, winner of **ILSVRC 2014**. Contains **inception modules**: stacking different types of convolutions. Uses **1×1 kernels** for dimensionality reduction.
- ▶ **ResNet** by He et al. [5], ResNet-152 won **ILSVRC 2015**. Introduces **residual connections** (later). State-of-the-art training of ResNet-50 in **224 seconds** by researchers at Sony is presented in Mikami et al. [11].

Others

- ▶ **OverFeat fast** and **OverFeat accurate** by Sermanet et al. [13] from 2013, won localization task in **ILSVRC 2013**, rank 4 for classification task, rank 1 for detection task, variants of AlexNet
- ▶ **VGG-16** by Simonyan and Zisserman [14], 13 convolutional layers, submitted to **ILSVRC 2014**. Developed by Oxford's Visual Geometry Group, weights and biases publically available, **replaces large kernels** (11×11 and 5×5) in AlexNet by several small 3×3 kernels.
- ▶ **GoogLeNet (Inception) v1** by Szegedy et al. [15], 22 layers, 2014, winner of **ILSVRC 2014**. Contains **inception modules**: stacking different types of convolutions. Uses **1×1 kernels** for dimensionality reduction.
- ▶ **ResNet** by He et al. [5], ResNet-152 won **ILSVRC 2015**. Introduces **residual connections** (later). State-of-the-art training of ResNet-50 in **224 seconds** by researchers at Sony is presented in Mikami et al. [11].

ILSVRC started in 2010 and **ended in 2017**. The winner in 2010 used an **SVM**, the winner in 2011 used a **kernel method** (Fisher vectors). ILSVRC announced a **3D competition** as replacement.

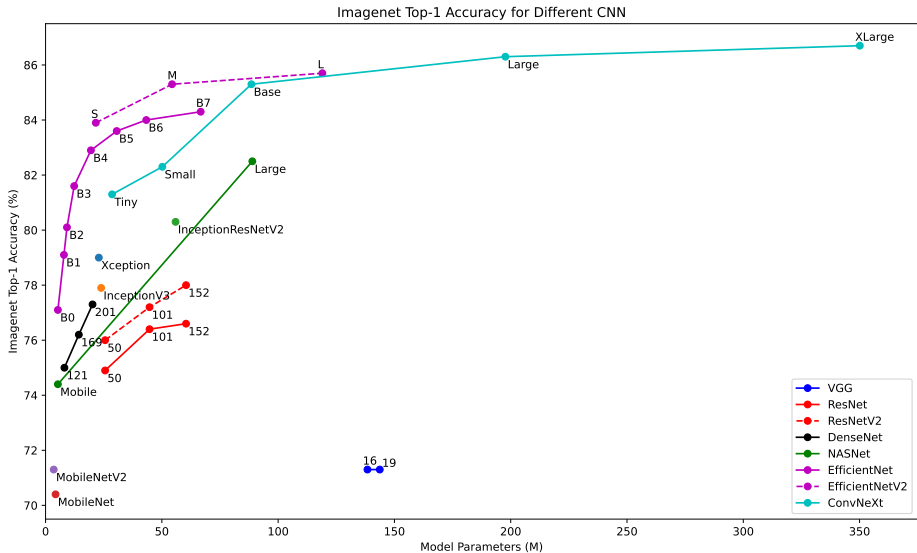
Others

- ▶ **OverFeat fast** and **OverFeat accurate** by Sermanet et al. [13] from 2013, won localization task in **ILSVRC 2013**, rank 4 for classification task, rank 1 for detection task, variants of AlexNet
- ▶ **VGG-16** by Simonyan and Zisserman [14], 13 convolutional layers, submitted to **ILSVRC 2014**. Developed by Oxford's Visual Geometry Group, weights and biases publically available, **replaces large kernels** (11×11 and 5×5) in AlexNet by several small 3×3 kernels.
- ▶ **GoogLeNet (Inception) v1** by Szegedy et al. [15], 22 layers, 2014, winner of **ILSVRC 2014**. Contains **inception modules**: stacking different types of convolutions. Uses **1×1 kernels** for dimensionality reduction.
- ▶ **ResNet** by He et al. [5], ResNet-152 won **ILSVRC 2015**. Introduces **residual connections** (later). State-of-the-art training of ResNet-50 in **224 seconds** by researchers at Sony is presented in Mikami et al. [11].

ILSVRC started in 2010 and **ended in 2017**. The winner in 2010 used an **SVM**, the winner in 2011 used a **kernel method** (Fisher vectors). ILSVRC announced a **3D competition** as replacement.

Later CNN focused on **reducing parameters**, we mention, e.g., **MobileNetV1** by Howard et al. [6] and **MobileNetV2** by Sandler et al. [12].

Overview of CNN (<https://keras.io/api/applications/>)



1D: Toeplitz to circulant

We rewrite the **full convolution** with the **Toeplitz matrix** $\mathbf{T}$

$$\begin{pmatrix} y_1 \\ \vdots \\ y_{k-1} \\ y_k \\ \vdots \\ y_n \\ y_{n+1} \\ \vdots \\ y_{n+k-1} \end{pmatrix} = \begin{pmatrix} f_1 & & & & & \\ \vdots & \ddots & & & & \\ f_{k-1} & \cdots & f_1 & & & \\ f_k & \cdots & f_2 & f_1 & & \\ & \ddots & \ddots & \ddots & \ddots & \\ & & f_k & \cdots & f_2 & f_1 \\ & & & f_k & \cdots & f_2 \\ & & & & \ddots & \vdots \\ & & & & & f_k \end{pmatrix} \begin{pmatrix} x_1 \\ \vdots \\ x_{k-1} \\ x_k \\ \vdots \\ x_n \end{pmatrix}, \quad \mathbf{T} \in \mathbb{R}^{(n+k-1) \times n}.$$

1D: Toeplitz to circulant

We rewrite the **full convolution** with the **Toeplitz matrix** $\mathbf{T}$

$$\begin{pmatrix} y_1 \\ \vdots \\ y_{k-1} \\ y_k \\ \vdots \\ y_n \\ y_{n+1} \\ \vdots \\ y_{n+k-1} \end{pmatrix} = \begin{pmatrix} f_1 & & & & & \\ \vdots & \ddots & & & & \\ f_{k-1} & \cdots & f_1 & & & \\ f_k & \cdots & f_2 & f_1 & & \\ & \ddots & \ddots & \ddots & \ddots & \\ & & f_k & \cdots & f_2 & f_1 \\ & & & f_k & \cdots & f_2 \\ & & & & \ddots & \vdots \\ & & & & & f_k \end{pmatrix} \begin{pmatrix} x_1 \\ \vdots \\ x_{k-1} \\ x_k \\ \vdots \\ x_n \\ 0 \\ \vdots \\ 0 \end{pmatrix}, \quad \mathbf{T} \in \mathbb{R}^{(n+k-1) \times (n+k-1)}.$$

1D: Toeplitz to circulant

We rewrite the **full convolution** with the **Toeplitz matrix** $\mathbf{T}$ and obtain a square **circulant matrix** $\mathbf{C}$:

$$\begin{pmatrix} y_1 \\ \vdots \\ y_{k-1} \\ y_k \\ \vdots \\ y_n \\ y_{n+1} \\ \vdots \\ y_{n+k-1} \end{pmatrix} = \begin{pmatrix} f_1 & & & & f_k & \cdots & f_2 \\ \vdots & \ddots & & & & \ddots & \vdots \\ f_{k-1} & \cdots & f_1 & & & & f_k \\ f_k & \cdots & f_2 & f_1 & & & \\ & \ddots & \ddots & \ddots & \ddots & & \\ & & f_k & \cdots & f_2 & f_1 & \\ & & & f_k & \cdots & f_2 & f_1 \\ & & & & \ddots & \vdots & \\ & & & & & f_k & f_{k-1} & \cdots & f_1 \end{pmatrix} \begin{pmatrix} x_1 \\ \vdots \\ x_{k-1} \\ x_k \\ \vdots \\ x_n \\ 0 \\ \vdots \\ 0 \end{pmatrix}, \quad \mathbf{C} \in \mathbb{R}^{(n+k-1) \times (n+k-1)}.$$

1D: Toeplitz to circulant

We rewrite the **full convolution** with the **Toeplitz matrix** $\mathbf{T}$ and obtain a square **circulant matrix** $\mathbf{C}$:

$$\begin{pmatrix} y_1 \\ \vdots \\ y_{k-1} \\ y_k \\ \vdots \\ y_n \\ y_{n+1} \\ \vdots \\ y_{n+k-1} \end{pmatrix} = \begin{pmatrix} f_1 & & & & f_k & \cdots & f_2 \\ \vdots & \ddots & & & & \ddots & \vdots \\ f_{k-1} & \cdots & f_1 & & & & f_k \\ f_k & \cdots & f_2 & f_1 & & & \\ & \ddots & \ddots & \ddots & \ddots & & \\ & & f_k & \cdots & f_2 & f_1 & \\ & & & f_k & \cdots & f_2 & f_1 \\ & & & & \ddots & \vdots & \\ & & & & & f_k & f_{k-1} & \cdots & f_1 \end{pmatrix} \begin{pmatrix} x_1 \\ \vdots \\ x_{k-1} \\ x_k \\ \vdots \\ x_n \\ 0 \\ \vdots \\ 0 \end{pmatrix}, \quad \mathbf{C} \in \mathbb{R}^{(n+k-1) \times (n+k-1)}.$$

We write the above relation **short-hand** as $\mathbf{y} = \mathbf{C}\mathbf{x}$, where $\mathbf{x}$ has been **extended by $k - 1$ zeros**.

1D: Fourier matrix and eigenvalues

The so-called **Fourier matrix $\mathbf{F}$** is the **complex symmetric** matrix of (transposed left and conjugate right) **eigenvectors** of **any circulant** matrix $\mathbf{C}$ ($\mathbf{FC} = \mathbf{\Lambda F}$),

$$\mathbf{C} = \begin{pmatrix} c_0 & c_{n-1} & \cdots & c_1 \\ c_1 & c_0 & \ddots & \vdots \\ \vdots & \ddots & \ddots & c_{n-1} \\ c_{n-1} & \cdots & c_1 & c_0 \end{pmatrix}, \quad \mathbf{F} = \begin{pmatrix} \omega^{0 \cdot 0} & \omega^{0 \cdot 1} & \cdots & \omega^{0 \cdot (n-1)} \\ \omega^{1 \cdot 0} & \omega^{1 \cdot 1} & \cdots & \omega^{1 \cdot (n-1)} \\ \vdots & \vdots & \ddots & \vdots \\ \omega^{(n-1) \cdot 0} & \omega^{(n-1) \cdot 1} & \cdots & \omega^{(n-1) \cdot (n-1)} \end{pmatrix}, \quad \omega = e^{-\frac{2\pi i}{n}}.$$

1D: Fourier matrix and eigenvalues

The so-called **Fourier matrix $\mathbf{F}$** is the **complex symmetric** matrix of (transposed left and conjugate right) **eigenvectors** of **any circulant** matrix $\mathbf{C}$ ($\mathbf{FC} = \mathbf{\Lambda F}$),

$$\mathbf{C} = \begin{pmatrix} c_0 & c_{n-1} & \cdots & c_1 \\ c_1 & c_0 & \ddots & \vdots \\ \vdots & \ddots & \ddots & c_{n-1} \\ c_{n-1} & \cdots & c_1 & c_0 \end{pmatrix}, \quad \mathbf{F} = \begin{pmatrix} \omega^{0 \cdot 0} & \omega^{0 \cdot 1} & \cdots & \omega^{0 \cdot (n-1)} \\ \omega^{1 \cdot 0} & \omega^{1 \cdot 1} & \cdots & \omega^{1 \cdot (n-1)} \\ \vdots & \vdots & \ddots & \vdots \\ \omega^{(n-1) \cdot 0} & \omega^{(n-1) \cdot 1} & \cdots & \omega^{(n-1) \cdot (n-1)} \end{pmatrix}, \quad \omega = e^{-\frac{2\pi i}{n}}.$$

This follows easily upon the **observation** that

$$\mathbf{C} = \sum_{j=0}^{n-1} \mathbf{P}^j c_j, \quad \mathbf{P} = \begin{pmatrix} 0 & \cdots & 0 & 1 \\ 1 & \ddots & \vdots & 0 \\ & \ddots & 0 & \vdots \\ & & 1 & 0 \end{pmatrix},$$

1D: Fourier matrix and eigenvalues

The so-called **Fourier matrix $\mathbf{F}$** is the **complex symmetric** matrix of (transposed left and conjugate right) **eigenvectors** of **any circulant** matrix $\mathbf{C}$ ($\mathbf{FC} = \mathbf{\Lambda F}$),

$$\mathbf{C} = \begin{pmatrix} c_0 & c_{n-1} & \cdots & c_1 \\ c_1 & c_0 & \ddots & \vdots \\ \vdots & \ddots & \ddots & c_{n-1} \\ c_{n-1} & \cdots & c_1 & c_0 \end{pmatrix}, \quad \mathbf{F} = \begin{pmatrix} \omega^{0 \cdot 0} & \omega^{0 \cdot 1} & \cdots & \omega^{0 \cdot (n-1)} \\ \omega^{1 \cdot 0} & \omega^{1 \cdot 1} & \cdots & \omega^{1 \cdot (n-1)} \\ \vdots & \vdots & \ddots & \vdots \\ \omega^{(n-1) \cdot 0} & \omega^{(n-1) \cdot 1} & \cdots & \omega^{(n-1) \cdot (n-1)} \end{pmatrix}, \quad \omega = e^{-\frac{2\pi i}{n}}.$$

This follows easily upon the **observation** that

$$\mathbf{C} = \sum_{j=0}^{n-1} \mathbf{P}^j c_j, \quad \mathbf{P} = \begin{pmatrix} 0 & \cdots & 0 & 1 \\ 1 & \ddots & \vdots & 0 \\ & \ddots & 0 & \vdots \\ & & 1 & 0 \end{pmatrix}, \quad \chi(z) = \det(z\mathbf{I} - \mathbf{P}) = z^n - 1.$$

1D: DFT & IDFT

Multiplication by the Fourier matrix is known as **discrete Fourier transformation (DFT)**,

$$\mathbf{F}\mathbf{x} = \hat{\mathbf{x}}, \quad \text{where} \quad \hat{x}_k = \sum_{j=0}^{n-1} e^{-\frac{2\pi i k j}{n}} x_j = \sum_{j=0}^{n-1} \omega^{kj} x_j = \sum_{j=0}^{n-1} (\omega^k)^j x_j, \quad k = 0, \dots, n-1.$$

1D: DFT & IDFT

Multiplication by the Fourier matrix is known as **discrete Fourier transformation (DFT)**,

$$\mathbf{F}\mathbf{x} = \hat{\mathbf{x}}, \quad \text{where} \quad \hat{x}_k = \sum_{j=0}^{n-1} e^{-\frac{2\pi i k j}{n}} x_j = \sum_{j=0}^{n-1} \omega^{kj} x_j = \sum_{j=0}^{n-1} (\omega^k)^j x_j, \quad k = 0, \dots, n-1.$$

The Fourier matrix has **orthogonal columns** of **length n** and is **symmetric**, so the inverse is

$$\mathbf{F}^{-1} = \frac{1}{n} \cdot \mathbf{F}^H = \frac{1}{n} \cdot \bar{\mathbf{F}}$$

1D: DFT & IDFT

Multiplication by the Fourier matrix is known as **discrete Fourier transformation (DFT)**,

$$\mathbf{F}\mathbf{x} = \hat{\mathbf{x}}, \quad \text{where} \quad \hat{x}_k = \sum_{j=0}^{n-1} e^{-\frac{2\pi i k j}{n}} x_j = \sum_{j=0}^{n-1} \omega^{kj} x_j = \sum_{j=0}^{n-1} (\omega^k)^j x_j, \quad k = 0, \dots, n-1.$$

The Fourier matrix has **orthogonal columns** of **length n** and is **symmetric**, so the inverse is

$$\mathbf{F}^{-1} = \frac{1}{n} \cdot \mathbf{F}^H = \frac{1}{n} \cdot \bar{\mathbf{F}}$$

and the **inverse DFT (IDFT)** is given by

$$\frac{1}{n} \bar{\mathbf{F}} \hat{\mathbf{x}} = \mathbf{x}, \quad \text{where} \quad x_k = \frac{1}{n} \sum_{j=0}^{n-1} e^{\frac{2\pi i k j}{n}} \hat{x}_j = \frac{1}{n} \sum_{j=0}^{n-1} \bar{\omega}^{kj} \hat{x}_j = \frac{1}{n} \sum_{j=0}^{n-1} (\bar{\omega}^k)^j \hat{x}_j, \quad k = 0, \dots, n-1.$$

1D: Convolution Theorem

We multiply $\mathbf{y} = \mathbf{C}\mathbf{x}$ by $\mathbf{F}$ and use the fact that $\mathbf{F}\mathbf{C} = \mathbf{\Lambda}\mathbf{F}$, where $\mathbf{\Lambda} = \text{diag}(\boldsymbol{\lambda})$ is the diagonal matrix of eigenvalues, to obtain

$$\hat{\mathbf{y}} = \mathbf{F}\mathbf{y} = \mathbf{F}\mathbf{C}\mathbf{x} = \mathbf{\Lambda}\mathbf{F}\mathbf{x} = \boldsymbol{\lambda} \circ \mathbf{F}\mathbf{x} = \boldsymbol{\lambda} \circ \hat{\mathbf{x}}.$$

1D: Convolution Theorem

We multiply $\mathbf{y} = \mathbf{C}\mathbf{x}$ by $\mathbf{F}$ and use the fact that $\mathbf{F}\mathbf{C} = \mathbf{\Lambda}\mathbf{F}$, where $\mathbf{\Lambda} = \text{diag}(\boldsymbol{\lambda})$ is the diagonal matrix of eigenvalues, to obtain

$$\hat{\mathbf{y}} = \mathbf{F}\mathbf{y} = \mathbf{F}\mathbf{C}\mathbf{x} = \mathbf{\Lambda}\mathbf{F}\mathbf{x} = \boldsymbol{\lambda} \circ \mathbf{F}\mathbf{x} = \boldsymbol{\lambda} \circ \hat{\mathbf{x}}.$$

The vector $\boldsymbol{\lambda}$ of eigenvalues is obtained using

$$\mathbf{C} = \sum_{j=0}^{n-1} c_j \mathbf{P}^j \Rightarrow \lambda_k = \sum_{j=0}^{n-1} (\omega^k)^j c_j = \hat{c}_k$$

since $\mathbf{P}$ has $\mathbf{e}_k^T \mathbf{F}$ as (left) eigenvectors to the eigenvalues ω^k , $k = 0, \dots, n-1$.

1D: Convolution Theorem

We multiply $\mathbf{y} = \mathbf{C}\mathbf{x}$ by $\mathbf{F}$ and use the fact that $\mathbf{FC} = \mathbf{\Lambda F}$, where $\mathbf{\Lambda} = \text{diag}(\boldsymbol{\lambda})$ is the diagonal matrix of eigenvalues, to obtain

$$\hat{\mathbf{y}} = \mathbf{F}\mathbf{y} = \mathbf{FC}\mathbf{x} = \mathbf{\Lambda F}\mathbf{x} = \boldsymbol{\lambda} \circ \mathbf{F}\mathbf{x} = \boldsymbol{\lambda} \circ \hat{\mathbf{x}}.$$

The vector $\boldsymbol{\lambda}$ of eigenvalues is obtained using

$$\mathbf{C} = \sum_{j=0}^{n-1} c_j \mathbf{P}^j \Rightarrow \lambda_k = \sum_{j=0}^{n-1} (\omega^k)^j c_j = \hat{c}_k$$

since $\mathbf{P}$ has $\mathbf{e}_k^T \mathbf{F}$ as (left) eigenvectors to the eigenvalues ω^k , $k = 0, \dots, n-1$.

Thus, the DFT maps convolution to elementwise multiplication,

$$\hat{\mathbf{y}} = \hat{\mathbf{c}} \circ \hat{\mathbf{x}} = \hat{\mathbf{f}} \circ \hat{\mathbf{x}} \Rightarrow \mathbf{y} = \text{IDFT}(\text{DFT}(\mathbf{f}) \circ \text{DFT}(\mathbf{x})),$$

where $\mathbf{c}$ is the (defining) first column of $\mathbf{C}$, here $\mathbf{c} = \mathbf{f}$ expanded by zeros. $\rightsquigarrow$ first Python example

1D convolution via DFT: NumPy

```
1 import numpy as np
2 from numpy.fft import fft, ifft
3
4 n, k = 10, 3
5 x = np.random.randint(0, 10, n)
6 f = np.random.randint(0, 10, k)
7
8 y = np.convolve(x, f)
9 print("Convolution_=\n", y)
10
11 x_pad = np.pad(x, (0, k-1))
12 f_pad = np.pad(f, (0, n-1))
13 y_dft = ifft(fft(f_pad) * fft(x_pad))
14 # y_dft has small imaginary components due to round-off
15 print("DFT_=\n", y_dft.real)
```

2D: convolution to block circulant w/ circulant blocks

“valid” convolution

$$\begin{pmatrix} y_{1,1} \\ y_{1,2} \\ y_{2,1} \\ y_{2,2} \end{pmatrix} = \begin{pmatrix} f_{2,2} & f_{2,1} & 0 \\ 0 & f_{2,2} & f_{2,1} \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} f_{1,2} & f_{1,1} & 0 \\ 0 & f_{1,2} & f_{1,1} \\ f_{2,2} & f_{2,1} & 0 \\ 0 & f_{2,2} & f_{2,1} \end{pmatrix} \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ f_{1,2} & f_{1,1} & 0 \\ 0 & f_{1,2} & f_{1,1} \end{pmatrix} \begin{pmatrix} x_{1,1} \\ x_{1,2} \\ x_{1,3} \\ x_{2,1} \\ x_{2,2} \\ x_{2,3} \\ x_{3,1} \\ x_{3,2} \\ x_{3,3} \end{pmatrix}$$

2D: convolution to block circulant w/ circulant blocks

“valid” convolution $\rightsquigarrow$ full convolution

$$\begin{pmatrix} y_{1,0} \\ y_{1,1} \\ y_{1,2} \\ y_{1,3} \\ y_{2,0} \\ y_{2,1} \\ y_{2,2} \\ y_{2,3} \end{pmatrix} = \begin{pmatrix} f_{2,1} & 0 & 0 \\ f_{2,2} & f_{2,1} & 0 \\ 0 & f_{2,2} & f_{2,1} \\ 0 & 0 & f_{2,2} \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} f_{1,1} & 0 & 0 \\ f_{1,2} & f_{1,1} & 0 \\ 0 & f_{1,2} & f_{1,1} \\ 0 & 0 & f_{1,2} \\ f_{2,1} & 0 & 0 \\ f_{2,2} & f_{2,1} & 0 \\ 0 & f_{2,2} & f_{2,1} \\ 0 & 0 & f_{2,2} \end{pmatrix} \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ f_{1,1} & 0 & 0 \\ f_{1,2} & f_{1,1} & 0 \\ 0 & f_{1,2} & f_{1,1} \\ 0 & 0 & f_{1,2} \end{pmatrix} \begin{pmatrix} x_{1,1} \\ x_{1,2} \\ x_{1,3} \\ x_{2,1} \\ x_{2,2} \\ x_{2,3} \\ x_{3,1} \\ x_{3,2} \\ x_{3,3} \end{pmatrix}$$

2D: convolution to block circulant w/ circulant blocks

“valid” convolution $\rightsquigarrow$ full convolution $\rightsquigarrow$ circulant blocks

$$\begin{pmatrix} y_{1,0} \\ y_{1,1} \\ y_{1,2} \\ y_{1,3} \\ y_{2,0} \\ y_{2,1} \\ y_{2,2} \\ y_{2,3} \end{pmatrix} = \begin{pmatrix} f_{2,1} & 0 & 0 & f_{2,2} & f_{1,1} & 0 & 0 & f_{1,2} & 0 & 0 & 0 & 0 \\ f_{2,2} & f_{2,1} & 0 & 0 & f_{1,2} & f_{1,1} & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & f_{2,2} & f_{2,1} & 0 & 0 & f_{1,2} & f_{1,1} & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & f_{2,2} & f_{2,1} & 0 & 0 & f_{1,2} & f_{1,1} & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & f_{2,1} & 0 & 0 & f_{2,2} & f_{1,1} & 0 & 0 & f_{1,2} \\ 0 & 0 & 0 & 0 & f_{2,2} & f_{2,1} & 0 & 0 & f_{1,2} & f_{1,1} & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & f_{2,2} & f_{2,1} & 0 & 0 & f_{1,2} & f_{1,1} & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & f_{2,2} & f_{2,1} & 0 & 0 & f_{1,2} & f_{1,1} \end{pmatrix} \begin{pmatrix} x_{1,1} \\ x_{1,2} \\ x_{1,3} \\ 0 \\ x_{2,1} \\ x_{2,2} \\ x_{2,3} \\ 0 \\ x_{3,1} \\ x_{3,2} \\ x_{3,3} \\ 0 \end{pmatrix}$$

2D: convolution to block circulant w/ circulant blocks

“valid” convolution $\rightsquigarrow$ full convolution $\rightsquigarrow$ circulant blocks $\rightsquigarrow$ block full convolution

$$\begin{pmatrix} y_{0,0} \\ y_{0,1} \\ y_{0,2} \\ y_{0,3} \\ y_{1,0} \\ \textcolor{red}{y_{1,1}} \\ \textcolor{red}{y_{1,2}} \\ y_{1,3} \\ y_{2,0} \\ \textcolor{red}{y_{2,1}} \\ \textcolor{red}{y_{2,2}} \\ y_{2,3} \\ y_{3,0} \\ y_{3,1} \\ y_{3,2} \\ y_{3,3} \end{pmatrix} = \begin{pmatrix} f_{1,1} & 0 & 0 & f_{1,2} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ f_{1,2} & f_{1,1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & f_{1,2} & f_{1,1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & f_{1,2} & f_{1,1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ f_{2,1} & 0 & 0 & f_{2,2} & f_{1,1} & 0 & 0 & f_{1,2} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ f_{2,2} & f_{2,1} & 0 & 0 & f_{1,2} & f_{1,1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & f_{2,2} & f_{2,1} & 0 & 0 & f_{1,2} & f_{1,1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & f_{2,2} & f_{2,1} & 0 & 0 & f_{1,2} & f_{1,1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & f_{2,1} & 0 & 0 & f_{2,2} & f_{1,1} & 0 & 0 & f_{1,2} & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & f_{2,2} & f_{2,1} & 0 & 0 & f_{1,2} & f_{1,1} & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & f_{2,2} & f_{2,1} & 0 & 0 & f_{1,2} & f_{1,1} & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & f_{2,2} & f_{2,1} & 0 & 0 & f_{1,2} & f_{1,1} & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & f_{2,1} & 0 & 0 & f_{2,2} & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & f_{2,2} & f_{2,1} & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & f_{2,2} & f_{2,1} & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & f_{2,2} & f_{2,1} & 0 & 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} x_{1,1} \\ x_{1,2} \\ x_{1,3} \\ 0 \\ x_{2,1} \\ x_{2,2} \\ x_{2,3} \\ 0 \\ x_{3,1} \\ x_{3,2} \\ x_{3,3} \\ 0 \end{pmatrix}$$

2D: convolution to block circulant w/ circulant blocks

“valid” convolution $\rightsquigarrow$ full convolution $\rightsquigarrow$ circulant blocks $\rightsquigarrow$ block full convolution $\rightsquigarrow$ block circulant

$$\begin{pmatrix} y_{0,0} \\ y_{0,1} \\ y_{0,2} \\ y_{0,3} \\ y_{1,0} \\ \textcolor{red}{y_{1,1}} \\ \textcolor{red}{y_{1,2}} \\ y_{1,3} \\ y_{2,0} \\ \textcolor{red}{y_{2,1}} \\ \textcolor{red}{y_{2,2}} \\ y_{2,3} \\ y_{3,0} \\ y_{3,1} \\ y_{3,2} \\ y_{3,3} \end{pmatrix} = \begin{pmatrix} f_{1,1} & 0 & 0 & f_{1,2} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & f_{2,1} & 0 & 0 & f_{2,2} \\ f_{1,2} & f_{1,1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & f_{2,2} & f_{2,1} & 0 & 0 \\ 0 & f_{1,2} & f_{1,1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & f_{2,2} & f_{2,1} & 0 \\ 0 & 0 & f_{1,2} & f_{1,1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & f_{2,2} & f_{2,1} \\ f_{2,1} & 0 & 0 & f_{2,2} & f_{1,1} & 0 & 0 & f_{1,2} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ f_{2,2} & f_{2,1} & 0 & 0 & f_{1,2} & f_{1,1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & f_{2,2} & f_{2,1} & 0 & 0 & f_{1,2} & f_{1,1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & f_{2,2} & f_{2,1} & 0 & 0 & f_{1,2} & f_{1,1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & f_{2,1} & 0 & 0 & f_{2,2} & f_{1,1} & 0 & 0 & f_{1,2} & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & f_{2,2} & f_{2,1} & 0 & 0 & f_{1,2} & f_{1,1} & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & f_{2,2} & f_{2,1} & 0 & 0 & f_{1,2} & f_{1,1} & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & f_{2,2} & f_{2,1} & 0 & 0 & f_{1,2} & f_{1,1} & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & f_{2,2} & f_{2,1} & 0 & 0 & 0 & f_{1,1} & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & f_{2,2} & f_{2,1} & 0 & 0 & 0 & f_{1,2} & f_{1,1} & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & f_{2,2} & f_{2,1} & 0 & 0 & 0 & f_{1,2} & f_{1,1} \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & f_{2,2} & f_{2,1} & 0 & 0 & 0 & f_{1,2} \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & f_{2,2} & f_{2,1} & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & f_{2,2} & f_{2,1} & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & f_{2,2} & f_{2,1} & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & f_{2,2} & f_{2,1} \end{pmatrix} \cdot \begin{pmatrix} x_{1,1} \\ x_{1,2} \\ x_{1,3} \\ 0 \\ x_{2,1} \\ x_{2,2} \\ x_{2,3} \\ 0 \\ x_{3,1} \\ x_{3,2} \\ x_{3,3} \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

2D: Kronecker product & block structure

We denote the **matrix of the last slide** by

$$\mathbf{C} := \begin{pmatrix} \mathbf{C}_0 & \mathbf{O} & \mathbf{O} & \mathbf{C}_1 \\ \mathbf{C}_1 & \mathbf{C}_0 & \mathbf{O} & \mathbf{O} \\ \mathbf{O} & \mathbf{C}_1 & \mathbf{C}_0 & \mathbf{O} \\ \mathbf{O} & \mathbf{O} & \mathbf{C}_1 & \mathbf{C}_0 \end{pmatrix}, \quad \mathbf{C}_0 := \begin{pmatrix} f_{1,1} & 0 & 0 & f_{1,2} \\ f_{1,2} & f_{1,1} & 0 & 0 \\ 0 & f_{1,2} & f_{1,1} & 0 \\ 0 & 0 & f_{1,2} & f_{1,1} \end{pmatrix}, \quad \mathbf{C}_1 := \begin{pmatrix} f_{2,1} & 0 & 0 & f_{2,2} \\ f_{2,2} & f_{2,1} & 0 & 0 \\ 0 & f_{2,2} & f_{2,1} & 0 \\ 0 & 0 & f_{2,2} & f_{2,1} \end{pmatrix}.$$

2D: Kronecker product & block structure

We denote the **matrix of the last slide** by

$$\mathbf{C} := \begin{pmatrix} \mathbf{C}_0 & \mathbf{O} & \mathbf{O} & \mathbf{C}_1 \\ \mathbf{C}_1 & \mathbf{C}_0 & \mathbf{O} & \mathbf{O} \\ \mathbf{O} & \mathbf{C}_1 & \mathbf{C}_0 & \mathbf{O} \\ \mathbf{O} & \mathbf{O} & \mathbf{C}_1 & \mathbf{C}_0 \end{pmatrix}, \quad \mathbf{C}_0 := \begin{pmatrix} f_{1,1} & 0 & 0 & f_{1,2} \\ f_{1,2} & f_{1,1} & 0 & 0 \\ 0 & f_{1,2} & f_{1,1} & 0 \\ 0 & 0 & f_{1,2} & f_{1,1} \end{pmatrix}, \quad \mathbf{C}_1 := \begin{pmatrix} f_{2,1} & 0 & 0 & f_{2,2} \\ f_{2,2} & f_{2,1} & 0 & 0 \\ 0 & f_{2,2} & f_{2,1} & 0 \\ 0 & 0 & f_{2,2} & f_{2,1} \end{pmatrix}.$$

We introduce the **Kronecker product of two matrices**, which is the block matrix

$$\mathbf{A} \otimes \mathbf{B} := \begin{pmatrix} a_{1,1}\mathbf{B} & \cdots & a_{1,n}\mathbf{B} \\ \vdots & \ddots & \vdots \\ a_{m,1}\mathbf{B} & \cdots & a_{m,n}\mathbf{B} \end{pmatrix},$$

2D: Kronecker product & block structure

We denote the **matrix of the last slide** by

$$\mathbf{C} := \begin{pmatrix} \mathbf{C}_0 & \mathbf{O} & \mathbf{O} & \mathbf{C}_1 \\ \mathbf{C}_1 & \mathbf{C}_0 & \mathbf{O} & \mathbf{O} \\ \mathbf{O} & \mathbf{C}_1 & \mathbf{C}_0 & \mathbf{O} \\ \mathbf{O} & \mathbf{O} & \mathbf{C}_1 & \mathbf{C}_0 \end{pmatrix}, \quad \mathbf{C}_0 := \begin{pmatrix} f_{1,1} & 0 & 0 & f_{1,2} \\ f_{1,2} & f_{1,1} & 0 & 0 \\ 0 & f_{1,2} & f_{1,1} & 0 \\ 0 & 0 & f_{1,2} & f_{1,1} \end{pmatrix}, \quad \mathbf{C}_1 := \begin{pmatrix} f_{2,1} & 0 & 0 & f_{2,2} \\ f_{2,2} & f_{2,1} & 0 & 0 \\ 0 & f_{2,2} & f_{2,1} & 0 \\ 0 & 0 & f_{2,2} & f_{2,1} \end{pmatrix}.$$

We introduce the **Kronecker product of two matrices**, which is the block matrix

$$\mathbf{A} \otimes \mathbf{B} := \begin{pmatrix} a_{1,1}\mathbf{B} & \cdots & a_{1,n}\mathbf{B} \\ \vdots & \ddots & \vdots \\ a_{m,1}\mathbf{B} & \cdots & a_{m,n}\mathbf{B} \end{pmatrix}, \quad \text{we have} \quad (\mathbf{A} \otimes \mathbf{B})(\mathbf{G} \otimes \mathbf{H}) = (\mathbf{A}\mathbf{G}) \otimes (\mathbf{B}\mathbf{H}).$$

2D: Kronecker product & block structure

We denote the **matrix of the last slide** by

$$\mathbf{C} := \begin{pmatrix} \mathbf{C}_0 & \mathbf{O} & \mathbf{O} & \mathbf{C}_1 \\ \mathbf{C}_1 & \mathbf{C}_0 & \mathbf{O} & \mathbf{O} \\ \mathbf{O} & \mathbf{C}_1 & \mathbf{C}_0 & \mathbf{O} \\ \mathbf{O} & \mathbf{O} & \mathbf{C}_1 & \mathbf{C}_0 \end{pmatrix}, \quad \mathbf{C}_0 := \begin{pmatrix} f_{1,1} & 0 & 0 & f_{1,2} \\ f_{1,2} & f_{1,1} & 0 & 0 \\ 0 & f_{1,2} & f_{1,1} & 0 \\ 0 & 0 & f_{1,2} & f_{1,1} \end{pmatrix}, \quad \mathbf{C}_1 := \begin{pmatrix} f_{2,1} & 0 & 0 & f_{2,2} \\ f_{2,2} & f_{2,1} & 0 & 0 \\ 0 & f_{2,2} & f_{2,1} & 0 \\ 0 & 0 & f_{2,2} & f_{2,1} \end{pmatrix}.$$

We introduce the **Kronecker product of two matrices**, which is the block matrix

$$\mathbf{A} \otimes \mathbf{B} := \begin{pmatrix} a_{1,1}\mathbf{B} & \cdots & a_{1,n}\mathbf{B} \\ \vdots & \ddots & \vdots \\ a_{m,1}\mathbf{B} & \cdots & a_{m,n}\mathbf{B} \end{pmatrix}, \quad \text{we have} \quad (\mathbf{A} \otimes \mathbf{B})(\mathbf{G} \otimes \mathbf{H}) = (\mathbf{A}\mathbf{G}) \otimes (\mathbf{B}\mathbf{H}).$$

Multiplication of the **circulant blocks** $\mathbf{C}_i$ by the Fourier matrix diagonalizes them, **$\mathbf{F}\mathbf{C}_i = \mathbf{\Lambda}_i\mathbf{F}$** .

2D: Kronecker product & block structure

We denote the **matrix of the last slide** by

$$\mathbf{C} := \begin{pmatrix} \mathbf{C}_0 & \mathbf{O} & \mathbf{O} & \mathbf{C}_1 \\ \mathbf{C}_1 & \mathbf{C}_0 & \mathbf{O} & \mathbf{O} \\ \mathbf{O} & \mathbf{C}_1 & \mathbf{C}_0 & \mathbf{O} \\ \mathbf{O} & \mathbf{O} & \mathbf{C}_1 & \mathbf{C}_0 \end{pmatrix}, \quad \mathbf{C}_0 := \begin{pmatrix} f_{1,1} & 0 & 0 & f_{1,2} \\ f_{1,2} & f_{1,1} & 0 & 0 \\ 0 & f_{1,2} & f_{1,1} & 0 \\ 0 & 0 & f_{1,2} & f_{1,1} \end{pmatrix}, \quad \mathbf{C}_1 := \begin{pmatrix} f_{2,1} & 0 & 0 & f_{2,2} \\ f_{2,2} & f_{2,1} & 0 & 0 \\ 0 & f_{2,2} & f_{2,1} & 0 \\ 0 & 0 & f_{2,2} & f_{2,1} \end{pmatrix}.$$

We introduce the **Kronecker product of two matrices**, which is the block matrix

$$\mathbf{A} \otimes \mathbf{B} := \begin{pmatrix} a_{1,1}\mathbf{B} & \cdots & a_{1,n}\mathbf{B} \\ \vdots & \ddots & \vdots \\ a_{m,1}\mathbf{B} & \cdots & a_{m,n}\mathbf{B} \end{pmatrix}, \quad \text{we have} \quad (\mathbf{A} \otimes \mathbf{B})(\mathbf{G} \otimes \mathbf{H}) = (\mathbf{AG}) \otimes (\mathbf{BH}).$$

Multiplication of the **circulant blocks** $\mathbf{C}_i$ by the Fourier matrix diagonalizes them, **$\mathbf{FC}_i = \mathbf{\Lambda}_i \mathbf{F}$** . As the matrix $\mathbf{C}$ is block circulant, the matrix $\mathbf{D}$ is a **block circulant** matrix with **diagonal blocks**,

$$(\mathbf{I} \otimes \mathbf{F})\mathbf{C} = \mathbf{D}(\mathbf{I} \otimes \mathbf{F}).$$

2D: DFT & IDFT in vectorized form

We multiply the **block circulant matrix** $\mathbf{D}$ from the left by $\mathbf{F} \otimes \mathbf{I}$,

$$(\mathbf{F} \otimes \mathbf{I})(\mathbf{I} \otimes \mathbf{F})\mathbf{C} = (\mathbf{F} \otimes \mathbf{I})\mathbf{D}(\mathbf{I} \otimes \mathbf{F}) = \mathbf{\Lambda}(\mathbf{F} \otimes \mathbf{I})(\mathbf{I} \otimes \mathbf{F}),$$

to obtain a **diagonal matrix** $\mathbf{\Lambda}$ of eigenvalues.

2D: DFT & IDFT in vectorized form

We multiply the **block circulant matrix $\mathbf{D}$** from the left by $\mathbf{F} \otimes \mathbf{I}$,

$$(\mathbf{F} \otimes \mathbf{I})(\mathbf{I} \otimes \mathbf{F})\mathbf{C} = (\mathbf{F} \otimes \mathbf{I})\mathbf{D}(\mathbf{I} \otimes \mathbf{F}) = \mathbf{\Lambda}(\mathbf{F} \otimes \mathbf{I})(\mathbf{I} \otimes \mathbf{F}),$$

to obtain a **diagonal matrix $\mathbf{\Lambda}$** of eigenvalues.

By the properties of the Kronecker product,

$$(\mathbf{F} \otimes \mathbf{I})(\mathbf{I} \otimes \mathbf{F}) = \mathbf{F} \otimes \mathbf{F} = (\mathbf{I} \otimes \mathbf{F})(\mathbf{F} \otimes \mathbf{I}),$$

2D: DFT & IDFT in vectorized form

We multiply the **block circulant matrix $\mathbf{D}$** from the left by $\mathbf{F} \otimes \mathbf{I}$,

$$(\mathbf{F} \otimes \mathbf{I})(\mathbf{I} \otimes \mathbf{F})\mathbf{C} = (\mathbf{F} \otimes \mathbf{I})\mathbf{D}(\mathbf{I} \otimes \mathbf{F}) = \mathbf{\Lambda}(\mathbf{F} \otimes \mathbf{I})(\mathbf{I} \otimes \mathbf{F}),$$

to obtain a **diagonal matrix $\mathbf{\Lambda}$** of eigenvalues.

By the properties of the Kronecker product,

$$(\mathbf{F} \otimes \mathbf{I})(\mathbf{I} \otimes \mathbf{F}) = \mathbf{F} \otimes \mathbf{F} = (\mathbf{I} \otimes \mathbf{F})(\mathbf{F} \otimes \mathbf{I}),$$

by structure this matrix diagonalizes **any block circulant w/ circulant blocks**.

2D: DFT & IDFT in vectorized form

We multiply the **block circulant matrix $\mathbf{D}$** from the left by $\mathbf{F} \otimes \mathbf{I}$,

$$(\mathbf{F} \otimes \mathbf{I})(\mathbf{I} \otimes \mathbf{F})\mathbf{C} = (\mathbf{F} \otimes \mathbf{I})\mathbf{D}(\mathbf{I} \otimes \mathbf{F}) = \mathbf{\Lambda}(\mathbf{F} \otimes \mathbf{I})(\mathbf{I} \otimes \mathbf{F}),$$

to obtain a **diagonal matrix $\mathbf{\Lambda}$** of eigenvalues.

By the properties of the Kronecker product,

$$(\mathbf{F} \otimes \mathbf{I})(\mathbf{I} \otimes \mathbf{F}) = \mathbf{F} \otimes \mathbf{F} = (\mathbf{I} \otimes \mathbf{F})(\mathbf{F} \otimes \mathbf{I}),$$

by structure this matrix diagonalizes **any block circulant w/ circulant blocks**.

The **inverse of $\mathbf{F}_n \otimes \mathbf{F}_m$** is given by $\frac{1}{mn} \bar{\mathbf{F}}_n \otimes \bar{\mathbf{F}}_m$,

2D: DFT & IDFT in vectorized form

We multiply the **block circulant matrix $\mathbf{D}$** from the left by $\mathbf{F} \otimes \mathbf{I}$,

$$(\mathbf{F} \otimes \mathbf{I})(\mathbf{I} \otimes \mathbf{F})\mathbf{C} = (\mathbf{F} \otimes \mathbf{I})\mathbf{D}(\mathbf{I} \otimes \mathbf{F}) = \mathbf{\Lambda}(\mathbf{F} \otimes \mathbf{I})(\mathbf{I} \otimes \mathbf{F}),$$

to obtain a **diagonal matrix $\mathbf{\Lambda}$** of eigenvalues.

By the properties of the Kronecker product,

$$(\mathbf{F} \otimes \mathbf{I})(\mathbf{I} \otimes \mathbf{F}) = \mathbf{F} \otimes \mathbf{F} = (\mathbf{I} \otimes \mathbf{F})(\mathbf{F} \otimes \mathbf{I}),$$

by structure this matrix diagonalizes **any block circulant w/ circulant blocks**.

The **inverse of $\mathbf{F}_n \otimes \mathbf{F}_m$** is given by $\frac{1}{mn}\bar{\mathbf{F}}_n \otimes \bar{\mathbf{F}}_m$, the **2D DFT and IDFT** are given in vectorized form by

$$\hat{\mathbf{x}} = (\mathbf{F}_n \otimes \mathbf{F}_m)\mathbf{x}, \quad \mathbf{x} = \frac{1}{mn}(\bar{\mathbf{F}}_n \otimes \bar{\mathbf{F}}_m)\hat{\mathbf{x}}.$$

2D: Convolution Theorem

The vectorized forms can be re-grouped to **matrix form**,

$$\mathbf{x} \rightsquigarrow \mathbf{X}, \quad \hat{\mathbf{x}} \rightsquigarrow \hat{\mathbf{X}}.$$

2D: Convolution Theorem

The vectorized forms can be re-grouped to **matrix form**,

$$\mathbf{x} \rightsquigarrow \mathbf{X}, \quad \hat{\mathbf{x}} \rightsquigarrow \hat{\mathbf{X}}.$$

Similar to the 1D case, the entries of the diagonal matrix Λ of eigenvalues of $\mathbf{C}$ are obtained by a **2D DFT of the zero-padded kernel**

2D: Convolution Theorem

The vectorized forms can be re-grouped to **matrix form**,

$$\mathbf{x} \rightsquigarrow \mathbf{X}, \quad \hat{\mathbf{x}} \rightsquigarrow \hat{\mathbf{X}}.$$

Similar to the 1D case, the entries of the diagonal matrix $\mathbf{\Lambda}$ of eigenvalues of $\mathbf{C}$ are obtained by a **2D DFT of the zero-padded kernel**, since the **eigenvalues of $\mathbf{A} \otimes \mathbf{B}$** are given by the products $\lambda_i(\mathbf{A}) \cdot \lambda_j(\mathbf{B})$ of the eigenvalues to the **eigenvectors $\mathbf{v}_i \otimes \mathbf{w}_j$** ,

$$(\mathbf{A} \otimes \mathbf{B})(\mathbf{v}_i \otimes \mathbf{w}_j) = (\mathbf{A}\mathbf{v}_i) \otimes (\mathbf{B}\mathbf{w}_j) = (\mathbf{v}_i \otimes \mathbf{w}_j)\lambda_i(\mathbf{A})\lambda_j(\mathbf{B}), \quad \lambda_{ij}(\mathbf{A} \otimes \mathbf{B}) := \lambda_i(\mathbf{A}) \cdot \lambda_j(\mathbf{B}),$$

2D: Convolution Theorem

The vectorized forms can be re-grouped to **matrix form**,

$$\mathbf{x} \rightsquigarrow \mathbf{X}, \quad \hat{\mathbf{x}} \rightsquigarrow \hat{\mathbf{X}}.$$

Similar to the 1D case, the entries of the diagonal matrix $\mathbf{\Lambda}$ of eigenvalues of $\mathbf{C}$ are obtained by a **2D DFT of the zero-padded kernel**, since the **eigenvalues of $\mathbf{A} \otimes \mathbf{B}$** are given by the products $\lambda_i(\mathbf{A}) \cdot \lambda_j(\mathbf{B})$ of the eigenvalues to the **eigenvectors $\mathbf{v}_i \otimes \mathbf{w}_j$** ,

$$(\mathbf{A} \otimes \mathbf{B})(\mathbf{v}_i \otimes \mathbf{w}_j) = (\mathbf{A}\mathbf{v}_i) \otimes (\mathbf{B}\mathbf{w}_j) = (\mathbf{v}_i \otimes \mathbf{w}_j)\lambda_i(\mathbf{A})\lambda_j(\mathbf{B}), \quad \lambda_{ij}(\mathbf{A} \otimes \mathbf{B}) := \lambda_i(\mathbf{A}) \cdot \lambda_j(\mathbf{B}),$$

and we can **diagonalize**

$$\mathbf{C} = \sum_{i,j} c_{i,j} \mathbf{P}^i \otimes \mathbf{P}^j \quad \text{by} \quad \mathbf{F} \otimes \mathbf{F}.$$

2D: Convolution Theorem

The vectorized forms can be re-grouped to **matrix form**,

$$\mathbf{x} \rightsquigarrow \mathbf{X}, \quad \hat{\mathbf{x}} \rightsquigarrow \hat{\mathbf{X}}.$$

Similar to the 1D case, the entries of the diagonal matrix $\mathbf{\Lambda}$ of eigenvalues of $\mathbf{C}$ are obtained by a **2D DFT of the zero-padded kernel**, since the **eigenvalues of $\mathbf{A} \otimes \mathbf{B}$** are given by the products $\lambda_i(\mathbf{A}) \cdot \lambda_j(\mathbf{B})$ of the eigenvalues to the **eigenvectors $\mathbf{v}_i \otimes \mathbf{w}_j$** ,

$$(\mathbf{A} \otimes \mathbf{B})(\mathbf{v}_i \otimes \mathbf{w}_j) = (\mathbf{A}\mathbf{v}_i) \otimes (\mathbf{B}\mathbf{w}_j) = (\mathbf{v}_i \otimes \mathbf{w}_j)\lambda_i(\mathbf{A})\lambda_j(\mathbf{B}), \quad \lambda_{ij}(\mathbf{A} \otimes \mathbf{B}) := \lambda_i(\mathbf{A}) \cdot \lambda_j(\mathbf{B}),$$

and we can **diagonalize**

$$\mathbf{C} = \sum_{i,j} c_{i,j} \mathbf{P}^i \otimes \mathbf{P}^j \quad \text{by} \quad \mathbf{F} \otimes \mathbf{F}.$$

Similar to the 1D case, the **2D DFT maps convolutions to elementwise multiplications**,

$$\mathbf{Y} = \mathbf{X} * \mathbf{F} \quad \Leftrightarrow \quad \hat{\mathbf{Y}} = \hat{\mathbf{F}} \circ \hat{\mathbf{X}}.$$

2D: DFT & IDFT in matrix form

The steps we took, **re-interpreted in matrix form**, are:

- ▶ **extending the input** from a matrix $\mathbf{X} \in \mathbb{R}^{n_1 \times n_2}$ by **zero-padding** to $\tilde{\mathbf{X}} \in \mathbb{R}^{(n_1+k_1-1) \times (n_2+k_2-1)}$,

$$\mathbf{X} = \begin{pmatrix} x_{1,1} & x_{1,2} & x_{1,3} \\ x_{2,1} & x_{2,2} & x_{2,3} \\ x_{3,1} & x_{3,2} & x_{3,3} \end{pmatrix} \rightsquigarrow \tilde{\mathbf{X}} = \begin{pmatrix} x_{1,1} & x_{1,2} & x_{1,3} & 0 \\ x_{2,1} & x_{2,2} & x_{2,3} & 0 \\ x_{3,1} & x_{3,2} & x_{3,3} & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix},$$

similarly for the kernel $\mathbf{K} \in \mathbb{R}^{k_1 \times k_2}$ to $\tilde{\mathbf{K}} \in \mathbb{R}^{(n_1+k_1-1) \times (n_2+k_2-1)}$,

2D: DFT & IDFT in matrix form

The steps we took, **re-interpreted in matrix form**, are:

- ▶ **extending the input** from a matrix $\mathbf{X} \in \mathbb{R}^{n_1 \times n_2}$ by **zero-padding** to $\tilde{\mathbf{X}} \in \mathbb{R}^{(n_1+k_1-1) \times (n_2+k_2-1)}$,

$$\mathbf{X} = \begin{pmatrix} x_{1,1} & x_{1,2} & x_{1,3} \\ x_{2,1} & x_{2,2} & x_{2,3} \\ x_{3,1} & x_{3,2} & x_{3,3} \end{pmatrix} \rightsquigarrow \tilde{\mathbf{X}} = \begin{pmatrix} x_{1,1} & x_{1,2} & x_{1,3} & 0 \\ x_{2,1} & x_{2,2} & x_{2,3} & 0 \\ x_{3,1} & x_{3,2} & x_{3,3} & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix},$$

similarly for the kernel $\mathbf{K} \in \mathbb{R}^{k_1 \times k_2}$ to $\tilde{\mathbf{K}} \in \mathbb{R}^{(n_1+k_1-1) \times (n_2+k_2-1)}$,

- ▶ **2D DFT** of the extended input and kernel, **componentwise multiplication**,

$$\hat{\mathbf{X}} = \mathbf{F}\tilde{\mathbf{X}}\mathbf{F}, \quad \hat{\mathbf{K}} = \mathbf{F}\tilde{\mathbf{K}}\mathbf{F}, \quad \hat{\mathbf{Y}} = \hat{\mathbf{K}} \circ \hat{\mathbf{X}},$$

2D: DFT & IDFT in matrix form

The steps we took, **re-interpreted in matrix form**, are:

- ▶ **extending the input** from a matrix $\mathbf{X} \in \mathbb{R}^{n_1 \times n_2}$ by **zero-padding** to $\tilde{\mathbf{X}} \in \mathbb{R}^{(n_1+k_1-1) \times (n_2+k_2-1)}$,

$$\mathbf{X} = \begin{pmatrix} x_{1,1} & x_{1,2} & x_{1,3} \\ x_{2,1} & x_{2,2} & x_{2,3} \\ x_{3,1} & x_{3,2} & x_{3,3} \end{pmatrix} \rightsquigarrow \tilde{\mathbf{X}} = \begin{pmatrix} x_{1,1} & x_{1,2} & x_{1,3} & 0 \\ x_{2,1} & x_{2,2} & x_{2,3} & 0 \\ x_{3,1} & x_{3,2} & x_{3,3} & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix},$$

similarly for the kernel $\mathbf{K} \in \mathbb{R}^{k_1 \times k_2}$ to $\tilde{\mathbf{K}} \in \mathbb{R}^{(n_1+k_1-1) \times (n_2+k_2-1)}$,

- ▶ **2D DFT** of the extended input and kernel, **componentwise multiplication**,

$$\hat{\mathbf{X}} = \mathbf{F}\tilde{\mathbf{X}}\mathbf{F}, \quad \hat{\mathbf{K}} = \mathbf{F}\tilde{\mathbf{K}}\mathbf{F}, \quad \hat{\mathbf{Y}} = \hat{\mathbf{K}} \circ \hat{\mathbf{X}},$$

- ▶ **2D IDFT** and **selection of region of interest** to obtain the convolution,

$$\tilde{\mathbf{Y}} = \frac{1}{(n+k-1)^2} \overline{\mathbf{F}} \hat{\mathbf{Y}} \mathbf{F}, \quad \mathbf{Y} = \tilde{\mathbf{Y}}[k-1:n, k-1:n].$$

$\rightsquigarrow$ second Python example

2D convolution via DFT: NumPy

```
1 import numpy as np
2 from scipy.signal import convolve2d
3 from scipy.linalg import dft
4
5 n, k = 8, 3
6 X = np.random.randint(0, 10, (n, n))
7 K = np.random.randint(0, 10, (k, k))
8 Y = convolve2d(X, K, mode='valid')
9 print("Convolution_=\n", Y)
10
11 X_pad = np.pad(X, ((0, k-1), (0, k-1)))
12 K_pad = np.pad(K, ((0, n-1), (0, n-1)))
13 F = dft(n+k-1)
14 X_hat = F @ X_pad @ F
15 K_hat = F @ K_pad @ F
16 Y_hat = X_hat * K_hat
17 Y_full = F.conj() @ Y_hat @ F.conj() / (n+k-1) ** 2
18 Y_dft = Y_full[k-1:n,k-1:n]
19 # Y_dft has small imaginary components due to round-off
20 print("DFT_=\n", Y_dft.real)
```

References I

- [1] Sharan Chetlur et al. “cuDNN: Efficient Primitives for Deep Learning”. In: *arXiv e-prints*, arXiv:1410.0759 (Oct. 2014). arXiv: 1410.0759 [cs.NE].
- [2] Dan Ciresan et al. “Flexible, High Performance Convolutional Neural Networks for Image Classification”. In: *Proceedings of the Twenty-Second International Joint Conference on Artificial Intelligence*. Vol. 2. 2011, pp. 1237–1242.
- [3] Dipankar Das et al. “Distributed Deep Learning Using Synchronous Stochastic Gradient Descent”. In: *arXiv e-prints*, arXiv:1602.06709 (Feb. 2016). arXiv: 1602.06709 [cs.DC].
- [4] Kunihiro Fukushima. “Neocognitron: A Self-organizing Neural Network Model for a Mechanism of Pattern Recognition Unaffected by Shift in Position”. In: *Biological Cybernetics* 36.4 (1980), pp. 193–202. DOI: 10.1007/BF00344251. URL: <https://www.cs.princeton.edu/courses/archive/spr08/cos598B/Readings/Fukushima1980.pdf>.
- [5] Kaiming He et al. “Deep Residual Learning for Image Recognition”. In: *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2016. DOI: 10.1109/CVPR.2016.90.

References II

- [6] Andrew G. Howard et al. “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications”. In: *arXiv e-prints*, arXiv:1704.04861 (Apr. 2017). arXiv: 1704.04861 [cs.CV].
- [7] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. “ImageNet Classification with Deep Convolutional Neural Networks”. In: *Advances in Neural Information Processing Systems 25*. Ed. by F. Pereira et al. Curran Associates, Inc., 2012, pp. 1097–1105. URL: <http://papers.nips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks.pdf>.
- [8] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. “ImageNet classification with deep convolutional neural networks”. In: *Communications of the ACM* 60.6 (2017), pp. 84–90. ISSN: 0001-0782. DOI: 10.1145/3065386.
- [9] Y. LeCun et al. “Backpropagation Applied to Handwritten Zip Code Recognition”. In: *Neural Computation* 1 (1989), pp. 541–551. URL: <http://yann.lecun.com/exdb/publis/pdf/lecun-89e.pdf>.

References III

- [10] Yann LeCun et al. “Gradient-based learning applied to document recognition”. In: *Proceedings of the IEEE* 86.11 (Nov. 1998), pp. 2278–2324. DOI: 10.1109/5.726791. URL: <http://yann.lecun.com/exdb/mnist/>.
- [11] Hiroaki Mikami et al. “Massively Distributed SGD: ImageNet/ResNet-50 Training in a Flash”. In: *arXiv e-prints*, arXiv:1811.05233 (Nov. 2018). arXiv: 1811.05233 [cs.LG].
- [12] Mark Sandler et al. “MobileNetV2: Inverted Residuals and Linear Bottlenecks”. In: *arXiv e-prints*, arXiv:1801.04381 (Jan. 2018). arXiv: 1801.04381 [cs.CV].
- [13] Pierre Sermanet et al. “OverFeat: Integrated Recognition, Localization and Detection using Convolutional Networks”. In: *arXiv e-prints*, arXiv:1312.6229 (Dec. 2013). arXiv: 1312.6229 [cs.CV].
- [14] Karen Simonyan and Andrew Zisserman. “Very Deep Convolutional Networks for Large-Scale Image Recognition”. In: *arXiv e-prints*, arXiv:1409.1556 (Dec. 2014). arXiv: 1409.1556 [cs.CV].

References IV

- [15] Christian Szegedy et al. “Going deeper with convolutions”. In: *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2015. DOI: 10.1109/CVPR.2015.7298594.
- [16] J Weng, N Ahuja, and TS Huang. “Learning recognition and segmentation of 3-D objects from 2-D images”. In: *Proc. 4th International Conf. Computer Vision*. 1993, pp. 121–128.
- [17] Kouichi Yamaguchi et al. “A Neural Network for Speaker-Independent Isolated Word Recognition”. In: *First International Conference on Spoken Language Processing (ICSLP 90)*. Kobe, Japan, Nov. 1990. URL: https://www.isca-speech.org/archive/icslp_1990/i90_1077.html.